

## MÓDULO 0 - AMBIENTE PROFISSIONAL

Ana Carolina da Silveira

### **O QUE É O EVENT LOOP DO NODE.JS E POR QUE ELE PERMITE ATENDER MUITAS CONEXÕES SIMULTÂNEAS COM UMA ÚNICA THREAD PRINCIPAL.**

É o mecanismo central que gerencia o fluxo de execução do Node.JS que gerencia a execução de tarefas assíncronas e callbacks em uma única thread (linha de execução) principal. Ele funciona como um ciclo infinito que verifica se há mensagens ou eventos pendentes para executar, permitindo que o Node.js processe milhares de conexões ao mesmo tempo sem travar.

### **QUAL É A DIFERENÇA ENTRE OPERAÇÕES SÍNCRONAS E ASSÍNCRONAS EM JAVASCRIPT, E POR QUE BLOQUEAR O EVENT LOOP É CONSIDERADO UM ERRO GRAVE EM PRODUÇÃO.**

As operações síncronas executam em sequência, enquanto operações assíncronas permitem que outras tarefas rodem em paralelo. A programação assíncrona possui algumas vantagens em relação à programação síncrona. Uma das principais vantagens é a capacidade de executar várias tarefas ao mesmo tempo, sem bloquear a execução do programa. Isso é especialmente útil em situações em que você precisa realizar operações que podem levar algum tempo.

O event loop é o mecanismo responsável por coordenar a execução do código JavaScript. Ele monitora a call stack (pilha de execução) e as filas de tarefas assíncronas, verificando continuamente se há funções prontas para serem executadas. Quando a pilha está vazia, o event loop move tarefas pendentes da fila para a call stack, permitindo que sejam processadas.

Bloquear o Event Loop é um erro grave porque o Node.js é single-threaded (possui apenas uma linha de execução principal) para o seu código JavaScript. Quando o Event Loop trava, toda a aplicação para de funcionar para todos os usuários ao mesmo tempo.

## **O QUE É TYPESCRIPT E QUAL PROBLEMA ELE RESOLVE EM RELAÇÃO AO JAVASCRIPT PURO (EXPLICAR A DIFERENÇA ENTRE TIPAGEM ESTÁTICA EM TEMPO DE COMPILAÇÃO E TIPAGEM DINÂMICA EM TEMPO DE EXECUÇÃO).**

O TypeScript é uma linguagem de programação criada pela Microsoft que funciona como um "superconjunto" do JavaScript, adicionando regras de tipos e ferramentas avançadas a ele. O JavaScript é uma linguagem muito flexível, mas essa flexibilidade traz problemas em projetos grandes.

- ★ Ele não verifica se você passou o tipo certo de dado para uma função ou variável.
- ★ Erros de tipo só aparecem na tela do usuário ou do servidor quando o código é executado.
- ★ Isso pode gerar falhas inesperadas no sistema em produção.

O TypeScript resolve isso permitindo que você defina os tipos de dados (como textos, números ou listas) antes de rodar o programa.

Tipagem Estática (Tempo de Compilação) - Na tipagem estática, os tipos das variáveis são definidos e verificados antes de o programa rodar, durante a compilação.

Tipagem Dinâmica (Tempo de Execução) - Na tipagem dinâmica, o tipo de dado de uma variável é definido pelo valor que ela recebe no momento em que o código está rodando.

## **QUAL É A DIFERENÇA ENTRE COMMONJS (REQUIRE/MODULE.EXPORTS) E ES MODULES (IMPORT/EXPORT) NO NODE.JS, E POR QUE ESSA DIFERENÇA AINDA GERA PROBLEMAS DE COMPATIBILIDADE EM PROJETOS REAIS.**

A principal diferença é que o CommonJS carrega os arquivos de forma síncrona e em tempo de execução, enquanto o ES Modules analisa e carrega o código de forma estática antes da execução começar.

Destaques do CommonJS (require e module.exports):

- ★ Síncrono: O código espera o arquivo terminar de carregar para continuar.
- ★ Dinâmico: Você pode chamar o require() dentro de condições, como um comando if.
- ★ Padrão antigo: Era o único jeito de dividir códigos em arquivos no Node.js por muitos anos.
- ★ Variáveis locais: Disponibiliza variáveis como \_\_dirname e \_\_filename automaticamente.

Destaques do ES Modules (import e export):

- ★ Estático: Os comandos import ficam sempre no topo do arquivo e são processados antes de o código rodar.
- ★ Assíncrono: Projetado para carregar arquivos de forma eficiente na web e no servidor.
- ★ Padrão moderno: É o padrão oficial da linguagem JavaScript para navegadores e versões novas do Node.js.
- ★ Sem variáveis locais: Não existem \_\_dirname ou \_\_filename de forma direta.

**PESQUISAR E COMPARAR, EM PELO MENOS UM PARÁGRAFO CADA, TRÊS FRAMEWORKS WEB CITADOS NA SEÇÃO 14.1 (EXPRESS, FASTIFY E NESTJS), INDICANDO UM CENÁRIO REAL EM QUE CADA UM SERIA A ESCOLHA MAIS INDICADA.**

★ Express.js

- Características Principais: Framework minimalista, não opinativo e tradicional. Oferece flexibilidade total para estruturar a arquitetura do projeto e conta com uma comunidade massiva, traduzida em um vasto ecossistema de middlewares.
- Desvantagens: A falta de padrões estruturais pode levar a códigos desorganizados e difíceis de manter em projetos de longo prazo ou com equipes grandes.
- Cenário Real Indicado: MVPs, Provas de Conceito (PoCs) e Microserviços simples. Ideal para situações onde a velocidade de entrega inicial é prioritária e a equipe precisa de uma ferramenta familiar e sem barreiras de entrada.

### ★ Fastify

- Características Principais: Focado em altíssimo desempenho, baixo overhead e eficiência de recursos. Utiliza compilação de esquemas JSON para serialização ultra-rápida, possui suporte nativo robusto a TypeScript e um sistema de arquitetura baseado em plugins encapsulados.
- Desvantagens: Ecossistema de bibliotecas de terceiros menor quando comparado ao Express, embora esteja em constante crescimento.
- Cenário Real Indicado: APIs de alta performance, plataformas de IoT e sistemas de chat/streaming em tempo real. Ideal para produtos de tecnologia que enfrentam volumes massivos de requisições simultâneas e precisam reduzir custos com infraestrutura de servidores (CPU e memória).

### ★ NestJS

- Características Principais: Framework fortemente opinativo, construído nativamente com TypeScript e inspirado na arquitetura do Angular. Utiliza conceitos de Programação Orientada a Objetos (POO), injeção de dependências e design patterns rigorosos (módulos, controllers e services). Funciona como uma camada robusta acima do Express ou Fastify.
- Desvantagens: Curva de aprendizado íngreme. Apresenta um nível de abstração elevado que pode ser considerado exagero para aplicações pequenas.
- Cenário Real Indicado: Aplicações corporativas (Enterprise) de grande porte, como sistemas bancários ou ERPs. Ideal para cenários onde múltiplos times de desenvolvimento trabalham no mesmo ecossistema e o projeto exige alta escalabilidade, manutenibilidade e padronização de código.

## **O QUE É UM GERENCIADOR DE VERSÕES DO NODE.JS (NVM) E POR QUE ELE É CONSIDERADO PRÁTICA OBRIGATÓRIA EM TIMES PROFISSIONAIS, CONFORME DISCUTIDO NA SEÇÃO 3.**

O NVM (Node Version Manager) é uma ferramenta de linha de comando que permite instalar, gerenciar e alternar entre diferentes versões do Node.js no mesmo computador. Ele é considerado uma prática obrigatória em times profissionais pelos seguintes motivos:

- ★ Incompatibilidade de projetos: Um projeto antigo da sua empresa pode rodar apenas no Node v14, enquanto o projeto novo exige o Node v20. O NVM permite alternar entre eles em segundos.
- ★ Ambientes idênticos: Ele garante que todos os desenvolvedores do time (e o servidor de produção) usem exatamente a mesma versão do Node.js, eliminando o erro clássico "na minha máquina funciona".
- ★ Automação com .nvmrc: É possível criar um arquivo chamado .nvmrc na raiz do projeto contendo o número da versão (ex: 18.16.0). Quando outro programador entra no projeto, ele digita apenas nvm use e o terminal muda para a versão correta automaticamente.
- ★ Permissões seguras: Instalar o Node.js globalmente direto do site oficial costuma exigir permissões de administrador (sudo no Mac/Linux). O NVM instala tudo na pasta do usuário, evitando problemas de permissão com pacotes globais do NPM.

## **O QUE É UM CONTAINER DOCKER E QUAL É A DIFERENÇA CONCEITUAL ENTRE UM CONTAINER E UMA MÁQUINA VIRTUAL.**

Um container Docker é um pacote leve, isolado e executável que reúne um aplicativo e tudo o que ele precisa para funcionar (como códigos, bibliotecas e arquivos de configuração).

- ★ Empacotamento completo: Junta o programa e suas dependências em um único lugar.
- ★ Portabilidade: Roda igual em qualquer computador que tenha o Docker instalado.
- ★ Independência: Funciona isolado de outros programas no mesmo computador.

## REFERÊNCIAS

O Loop de Eventos do Node.js. NodeJs. Disponível em: <https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick#the-nodejs-event-loop>. Acesso em 28 de Ago de 2026.

Paulo Clemente. Node.js: Entendendo O Event Loop. Rocketseat. Disponível em: <https://www.rocketseat.com.br/blog/artigos/post/node-js-event-loop>. Acesso em 28 de Ago de 2026.

Renan Lima. Assíncrona ou síncrona? Alura, 2023. Disponível em: <https://cursos.alura.com.br/forum/topico-duvida-assincrona-ou-sincrona-308537>. Acesso em 28 de Ago de 2026.

Ana Júlia Souza. Assincronismo e Event Loop em JavaScript. Gran, 2026. Disponível em: <https://blog.grancursosonline.com.br/assincronismo-e-event-loop-em-javascript/>. Acesso em 28 de Ago de 2026.

Redação Impacta. Typescript x JavaScript: entenda o que é e saiba as diferenças. Impacta. Disponível em: <https://www.impacta.com.br/blog/typescript-javascript-entenda-que-saiba-diferencas/>. Acesso em 28 de Ago de 2026.

Cristiano Albano. Conheça a diferença entre JavaScript e TypeScript. Coodesh. Disponível em: <https://coodesh.com/blog/candidates/conheca-a-diferenca-entre-javascript-e-typescript/>. Acesso em 28 de Ago de 2026.

Elton Fonseca. Quais as diferenças entre tipagens: estática ou dinâmica e forte ou fraca. Treinaweb, 2019. Disponível em: <https://www.treinaweb.com.br/blog/quais-as-diferencas-entre-tipagens-estatica-ou-dinamica-e-forte-ou-fraca>. Acesso em 28 de Ago de 2026.

Kingsley Ubah. Módulos CommonJS versus módulos ES no Node.js. LogRocket, 2024. Disponível em: <https://blog.logrocket.com/commonjs-vs-es-modules-node-js/>. Acesso em 28 de Ago de 2026.

CommonJS vs ES Modules: require ou import no Node? Caddy Docs. Disponível em: <https://caddy.tech/docs/pt/javascript/commonjs-vs-esm>. Acesso em 28 de Ago de 2026.

Igor Pestana. Comparando Frameworks para Node.js: Express, Fastify e Nest.js. Dev, 2024. Disponível em: <https://dev.to/igorspestana/comparando-frameworks-para-nodejs-express-fastify-e-nestjs-4een>. Acesso em 28 de Ago de 2026.

Fastify vs Express vs NestJS: O Stack Back-end que Faz API Escalar. CrazyStack. Disponível em: <https://www.crazystack.com.br/blog/meu-setup-de-dev-em-2026-o-que>. Acesso em 28 de Ago de 2026.

Shahid Islam. Express vs Koa vs Fastify vs NestJS vs Hono: Como escolher o framework Node.js certo. Medium, 2025. Disponível em: <https://medium.com/@khanshahid9283/express-vs-koa-vs-fastify-vs-nestjs-vs-hono-choosing-the-right-node-js-framework-17a56a533d29>. Acesso em 28 de Ago de 2026.

Luiz Ricardo Silva. Gerenciando versões do Node.js com NVM. Dev, 2023. Disponível em: <https://dev.to/oricardos/gerenciando-versoes-do-nodejs-com-nvm-cjc>. Acesso em 28 de Ago de 2026.

Andrew Cavalcante. O que é NVM? Guia Completo do Node Version Manager. ProgramAi, 2024. Disponível em: <http://programai.com.br/o-que-e-nvm/>. Acesso em 28 de Ago de 2026.

Qual é a diferença entre o Docker e uma VM?. AWS. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-docker-vm/>. Acesso em 28 de Ago de 2026.